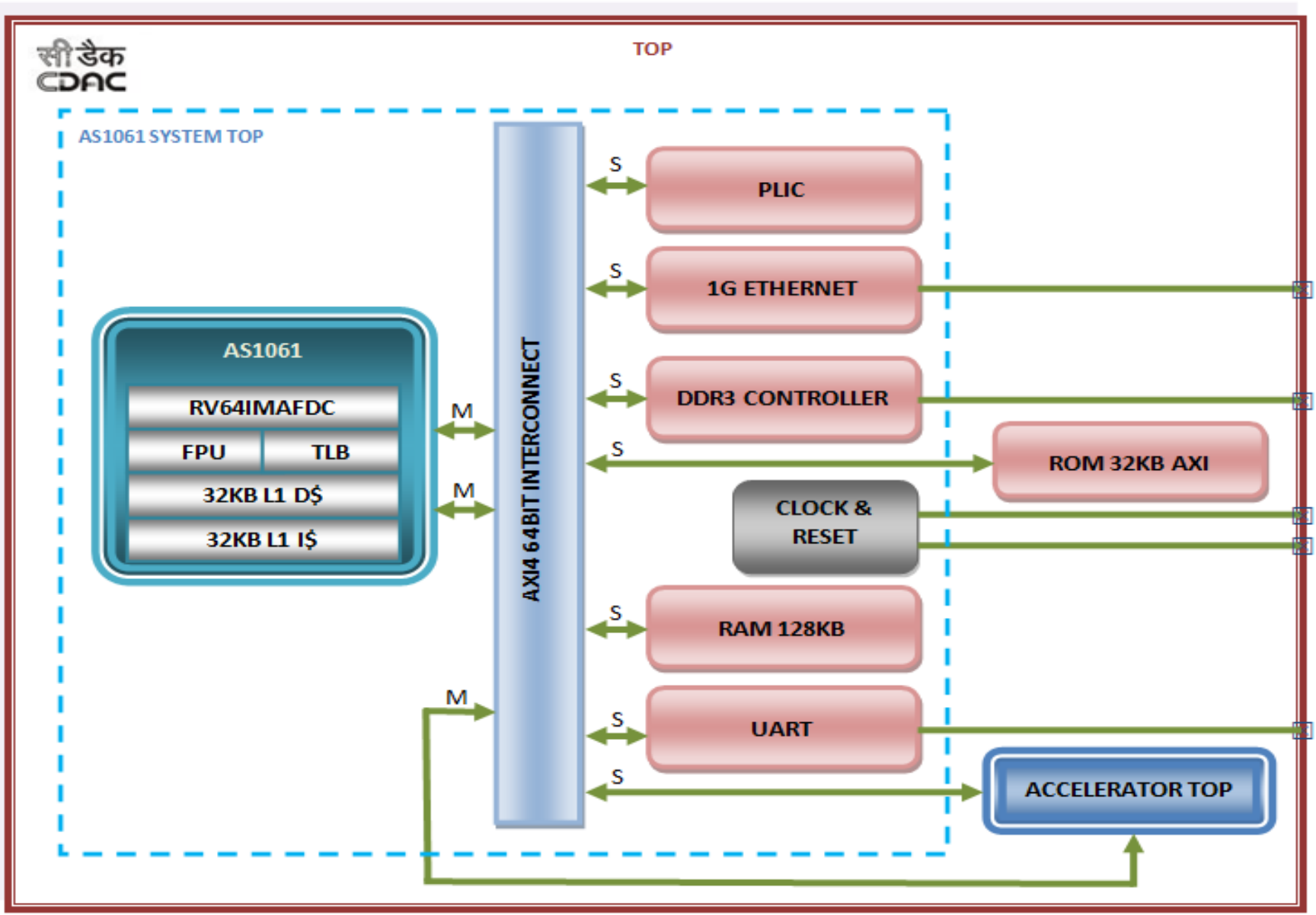
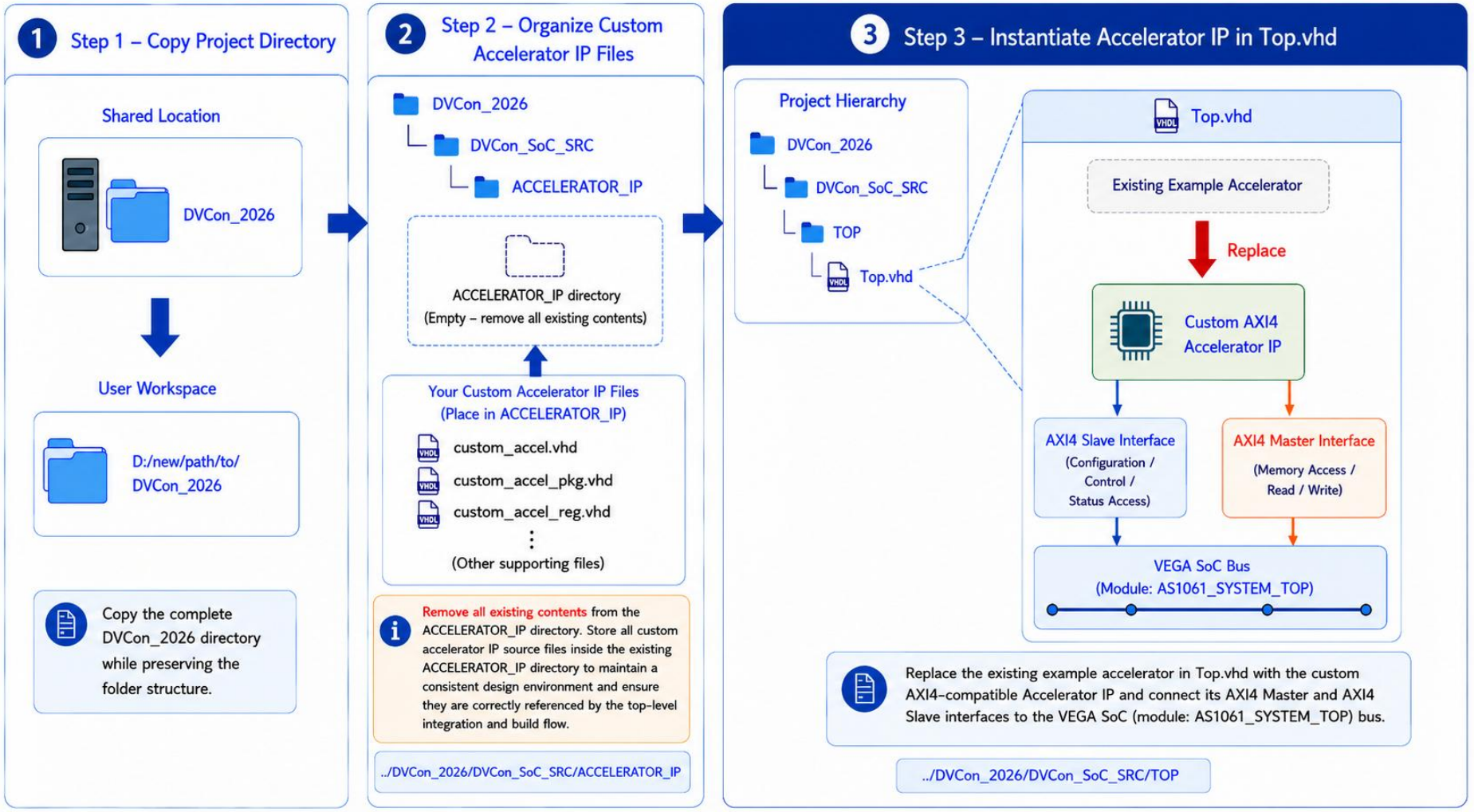

TOP
AS1061 SYSTEM TOP
AS1061
RV64IMAFDC
FPU
TLB
32KB L1 D$
32KB L1 I$
M
M
M
AXI4 64BIT INTERCONNECT
S
PLIC
S
1G ETHERNET
S
DDR3 CONTROLLER
S
ROM 32KB AXI
CLOCK & RESET
S
RAM 128KB
S
UART
S
ACCELERATOR TOP

Guide to Simulation and Implementation of DVCON Project in Vivado

Integrating a custom Accelerator IP into the VEGA SoC requires updating the top-level design, replacing the example accelerator, and connecting the new IP to the required SoC interfaces. Follow the steps below to complete the integration, simulation, and implementation flow.

Guide to Integrating a custom Accelerator IP into the VEGA SoC



Guide to Use run.sh Script

This script automates and streamlines your FPGA design workflow by presenting a simple terminal-based menu. It internally runs Vivado/QuestaSim TCL scripts for simulation, implementation, and FPGA programming.

1 Step 1: Execute the Script

- Run the script with:

```
./run.sh
```




```
user@linux:~/DVCon_2026/RUN

user@linux:~/DVCon_2026/RUN$ ./run.sh
```

2 Step 2: Choose an Option from the Menu

- Select the desired operation by entering the corresponding number.

```
user@linux:~/DVCon_2026/RUN
==========================================
            FPGA Project Tool Menu
Main Directory:  /your/path/here
==========================================
1) Simulation
2) FPGA Implementation
3) FPGA Programming
4) Exit
==========================================
Enter choice [1-4]:
```

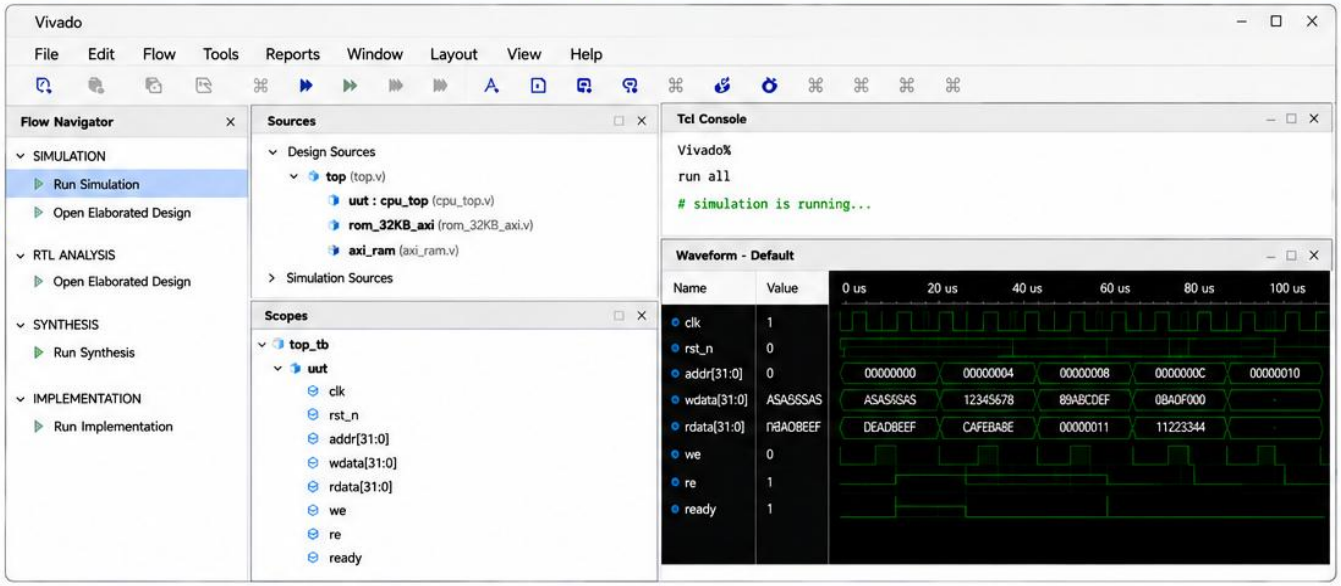
3 Step 3: Run Simulation (Option 1)

- Select the desired operation by entering the corresponding number.

```
user@linux:~/DVCon_2026/RUN
------------------------------------
       Choose Simulation Type
------------------------------------
1) Vivado Simulation
2) QuestaSim Simulation
------------------------------------
Enter choice [1-2]:
```

Vivado Simulation (Option 1)

```
user@linux:~/DVCon_2026/RUN
Running Vivado Simulation...
```



Follow These Steps After Vivado Launches:

1 Monitor the Simulation:

- Monitor the *Vivado Tcl Console* to track the progress.
- Wait for Vivado to enter simulation mode and open the waveform viewer.

2 Update the .mif (Program image) File:

../DVCon_2026/
DVCon_SoC_SRC/
MEMORY_IP/
rom_32KB_axi/

rom_32KB_axi.mif

- Replace the existing rom_32KB_axi.mif file with your program image (same filename).
- After replacing the file, relaunch the simulation using the Relaunch Simulation button in the toolbar or Run → Relaunch Simulation in Vivado for the changes to take effect.

3 Add Signals to the Waveform Viewer:

Scopes / Objects

- In *Scopes* or *Objects*, select signals and right-click "Add to Wave Window".

Right-click and
Add to Wave Window

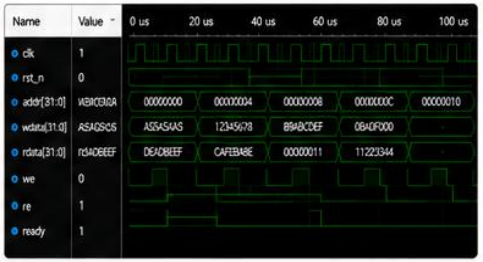
Waveform Viewer

4 Run the Simulation:

```
run 1000 us
```

- Simulate for 1000 us.
- Modify duration as needed (e.g., *run 500 us, run all*).

5 View Simulation Results:

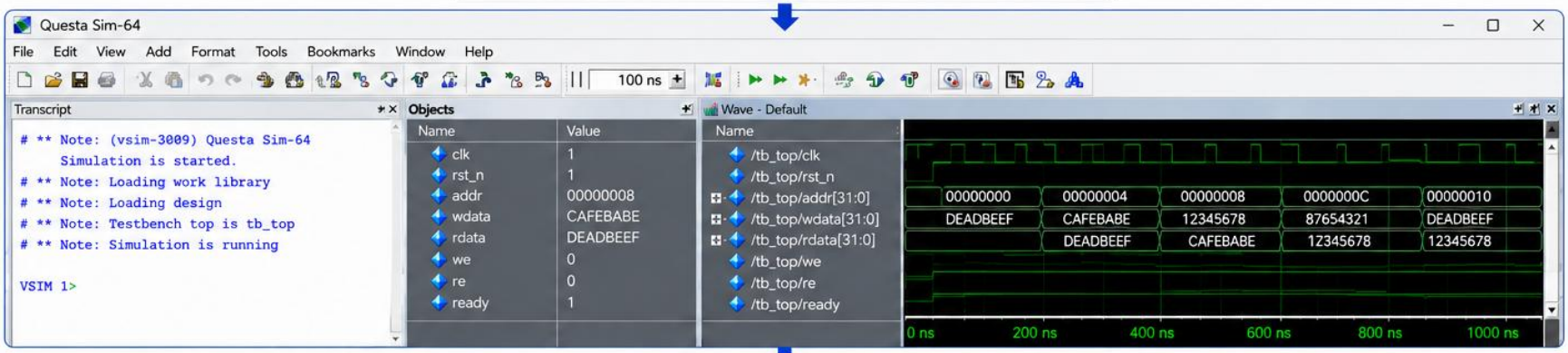


- Observe signal activity in the waveform viewer.
- Confirm whether the design functions as expected.

Verify correct design functionality.

QuestaSim Simulation (Option 2)

```
user@linux:~/DVCon_2026/RUN
Running QuestaSim Simulation...
```



1 Monitor Simulation

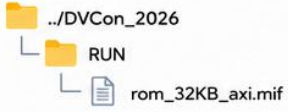
- Observe Transcript window
- Wait for testbench initialization

2 Add Signals

Objects / Signals → Add to Wave → Selected Signals → Wave Window

- Select the required signals and choose **Add to Wave → Selected Signals.**

3 Update the .mif (Program image) File



Program File → rom_32KB_axi.mif

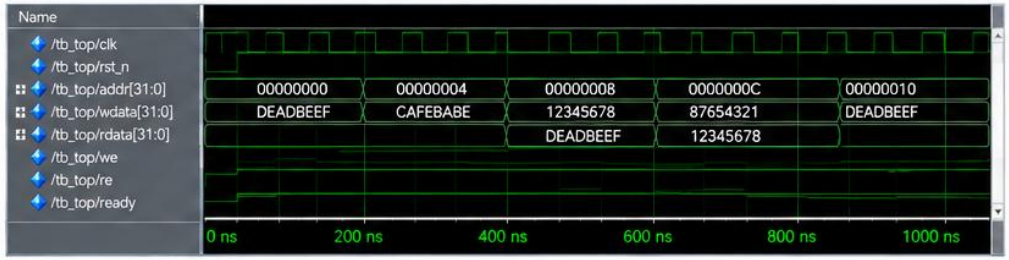
- Replace the existing rom_32KB_axi.mif file in this folder with your program image, ensuring that the filename remains **rom_32KB_axi.mif.**
- This file is used to pre-load the ROM during simulation.

4 Run Simulation

```
run 1000 us
```

- Simulation duration can be modified (**run 500 us, run all**, etc.)

5 View Results



Waveform Window

Signal Activity

Verify correct design functionality.

Run FPGA Implementation (Option 2)

1 Step 1: Choose an Option from the Menu

```
========================================
        FPGA Project Tool Menu
  Main Directory: /your/path/here
========================================
 1) Simulation
 2) FPGA Implementation      <-- selected
 3) FPGA Programming
 4) Exit
========================================
Enter choice [1-4]:
```

- Select the desired operation by entering the corresponding number.

2 Step 2: FPGA Implementation Options

```
----------------------------------------
      FPGA Implementation Options
----------------------------------------
1) Synthesis Only          <-- selected
2) Full Implementation
   (Synthesis + Implementation + Bitstream)
----------------------------------------

Enter choice [1-2]:
```

- Select the desired operation by entering the corresponding number.

```
Running Synthesis Only...
```

3 Check the Terminal Output

- Monitor the terminal while the synthesis script executes.
- Review progress messages, warnings, and error messages.

```
Terminal
user@linux:~/DVCon_2026/RUN$ ./run_impl.sh
Launching Vivado...
Running synthesis...
INFO: [Synth 8-3331] Analyzing design...
INFO: [Synth 8-5857] Optimizing design...
INFO: [Synth 8-2565] Generating reports...
INFO: [Common 17-206] Writing report file...
Completed.
```

4 Verify Results

Project Directory (Vivado Project)

```
DVCon_2026
└── VIVADO_PROJECT
    └── DVCon_pjt.xpr
```

(Vivado Project File)

Verify that the Vivado project was successfully created or updated.

Output Directory (Synthesis Output)

```
DVCon_2026
└── GEN_BIT_OUT
    ├── utilization.rpt
    ├── timing_summary.rpt
    └── ... (other synthesis reports)
```

- Confirm synthesis output files were generated.
- Review the .rpt files to analyze FPGA synthesis resource utilization.

5 Review Synthesis Reports

```
REPORT NAVIGATOR
▼ Synthesis Report
    Summary
    Utilization
    Timing
▶   Design Hierarchy
    Messages
```

Synthesis – Resource Utilization

Resource	Used	Available	Utilization	Utilization %
LUT	12,345	112,800		10.95 %
Flip-Flops	8,765	225,600		3.88 %
BRAM	25	150		16.67 %
DSP	10	240		4.17 %
IO	45	300		15.00 %

Timing Summary

Worst Negative Slack (WNS)	2.135 ns
Total Negative Slack (TNS)	0.000 ns
Number of Failing Endpoints	0
Total Power	0.456 W

Review synthesis results and verify successful completion.

Run FPGA Implementation (Option 2)

1 Choose an Option from the Menu

```
==========================================
       FPGA Project Tool Menu
  Main Directory: /your/path/here
==========================================
 1) Simulation
 2) FPGA Implementation
 3) FPGA Programming
 4) Exit
==========================================
Enter choice [1-4]:
```

- Select the desired operation by entering the corresponding number.

2 FPGA Implementation Options

```
------------------------------------------
       FPGA Implementation Options
------------------------------------------
1) Synthesis Only
2) Full Implementation
   (Synthesis + Implementation + Bitstream)
------------------------------------------
Enter choice [1-2]:
```

- Select the desired operation by entering the corresponding number.

```
Running Full Implementation...
```

3 Check the Terminal Output

Vivado runs in batch mode and executes all implementation steps defined in the DVCon_IMP.tcl script.

- These steps typically include:
  - ✔ Synthesis of RTL code
  - ✔ Implementation (placement and routing)
  - ✔ Bitstream generation
- You can monitor the progress and messages directly in the terminal or command prompt.
- The script will execute each step sequentially and provide feedback, including any warnings or errors encountered during execution.

```
Terminal
user@linux:~/DVCon_2026/RUN$ ./run_impl.sh
Launching Vivado...
Running full implementation...
INFO: [Synth 8-3331] Analyzing design...
INFO: [Synth 8-5857] Optimizing design...
INFO: [Impl 11-20] Placing design...
INFO: [Impl 11-30] Routing design...
INFO: [Impl 11-40] Generating bitstream...
INFO: [Common 17-206] Writing report files...
Completed.
```

4 Verify Results

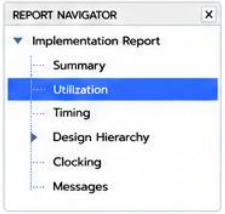
- **Project Directory:**
  To ensure the Vivado project was successfully created or updated.



- **Output Directory:**
  - Ensure the .bit file is present and has a non-zero file size, confirming successful bitstream generation.
  - Open the .rpt files in a text editor and review their contents for any warnings or errors that may indicate issues during synthesis or implementation.



5 Review Implementation Reports



Resource Utilization

Resource	Used	Available	Utilization	Utilization %
LUT	45,678	114,240		39.98 %
Flip-Flops	68,912	228,480		30.16 %
BRAM	112	312		35.90 %
DSP	248	900		27.56 %
BUFG	16	32		50.00 %

Timing Summary

Worst Negative Slack (WNS)	0.512 ns
Total Negative Slack (TNS)	0.000 ns
Number of Failing Endpoints	0
Total Power	1.256 W

Review implementation results and verify successful completion.

Run FPGA Programming (Option 3)

Before you can program your FPGA, make sure the .bit file is already generated. This file is created after completing FPGA Implementation (Option 2).




```
========================================
        FPGA Project Tool Menu
  Main Directory: /your/path/here
========================================
1) Simulation
2) FPGA Implementation
3) FPGA Programming
4) Exit
========================================
Enter choice [1-4]:
```

- The terminal will display:

```
" Programming FPGA..."
```

1 Program the FPGA:

- The script will automatically detect the connected device and load the bitstream.
- Watch the Vivado Tcl Console for messages like:

```
INFO: [Labtools 27-1434]
Device xc7z020 successfully programmed.
```

Example: Vivado Tcl Console Output

Vivado Tcl Console

```
Programming FPGA...
Open Hardware Manager...
Connecting to hardware server...
Auto-detected device: xc7z020clg400-1
Loading bitstream: ../DVCon_2026/GEN_BIT_OUT/DVCon_2026.bit
Programming device...
INFO: [Labtools 27-1434] Device xc7z020 successfully programmed.
Programming completed successfully.
```

2 Verify Programming via Serial Terminal:

- To open Minicom with the preset configuration, open a terminal window and run the following command:

```
$ sudo minicom dvcon
```

- Once the FPGA is successfully programmed, the Minicom terminal application should display the bootloader output, confirming that the device is up and running.

Example: Minicom Terminal Output

/dev/ttyUSB0 - 115200 8N1 - Minicom

```
+  +-----------------------------------------------------+
|  |  VEGA Series of Microprocessors Developed By C-DAC, India  |
|  | Digital India RISC-V (DIR-V) Program, MeitY, Govt, of India |
+  +-----------------------------------------+-------------+
|  |                    DVCON 2026           |    ||       |
+  +-----------------------------+-----------+-------------+
|  |   www.vegaprocessors.in     |        vega@cdac.in     |
+  +-----------------------------+-------------------------+

MAC address     : 00:A0:22:01:01:80
Initializing ethernet...
Waiting for data.......
```

Exit the Script (Option 4)

1 Step 1: Choose an Option from the Menu

- Select **Option 4 – Exit** from the FPGA Project Tool Menu.
- Use this option to close the tool after simulation, implementation, or FPGA programming, or to exit without performing any operation.

```
==========================================
          FPGA Project Tool Menu
     Main Directory: /your/path/here
==========================================
 1) Simulation
 2) FPGA Implementation
 3) FPGA Programming
 4) Exit
==========================================
Enter choice [1-4]:
```

Accelerator Software Development and Linux Integration

1 Modify the Application

DVCon_2026/
- Application/
  - docker
  - buildroot_dvcon
  - build
  - drv_test_128.c
  - app_compile_copy.sh
  - ...

- Add or modify your application in the drv_test_128.c file.
- Rebuild the application after making changes.

drv_test_128.c

```c
int main(void)
{
    // User application
    accelerator_test_128();
    return 0;
}
```

2 Compile the Application

DVCon_2026/
- Application/
  - docker
  - buildroot_dvcon
  - build
  - drv_test_128.c
  - app_compile_copy.sh
  - ...

```
$ ./app_compile_copy.sh
```

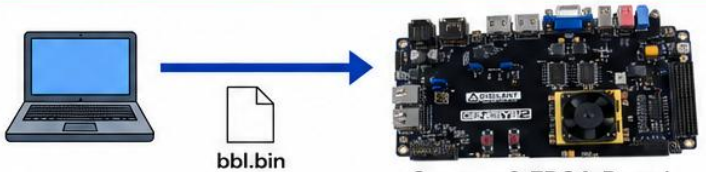
- The script automatically compiles the application, integrates it into the Linux image, and transfer to the output file to FILE_TRANSFER folder.

```
File  Edit  View  Search  Terminal  Help
[hdg@hdg_vivado-lab4 Application]$ ./app_compile_copy.sh
Checking for Docker...
[*] Docker image 'buildroot-2004' already exists. Skipping build.
Compiling app...
Copying to Linux
>>> skeleton-init-common  Installing to target
rsync -a --ignore-times --exclude .svn --exclude .git --exclude .hg --exclude .bzr --exclude CVS --chmod=u+rwx
.go=rx --exclude .empty --exclude '.* ' system/skeleton/ /workspace/buildroot/output/target/
/usr/bin/install -d -m 0755 /workspace/buildroot/output/target/bin
/usr/bin/install -d -m 0755 /workspace/buildroot/output/target/sbin
/usr/bin/install -d -m 0755 /workspace/buildroot/output/target/lib
ln -snf lib /workspace/buildroot/output/target/lib64
ln -snf lib /workspace/buildroot/output/target/usr/lib64
/usr/bin/sed -i -e 's,@PATH@,"/bin:/sbin:/usr/bin:/usr/sbin",' /workspace/buildroot/output/target/etc/profile
/usr/bin/install -m 0644 support/misc/target-dir-warning.txt /workspace/buildroot/output/target/THIS_IS_NOT_YO
```

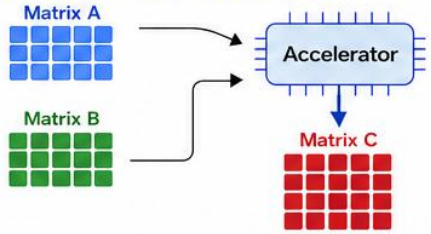
3 Generated Output

DVCon_2026/
- FILE_TRANSFER/
  - bbl.bin
  - FDC.dts
  - send.sh
  - ...

- You can send Linux binary file (bbl.bin) to FPGA board.



4 Default Example Application



- Matrix A and Matrix B are read from memory.
- Hardware accelerator performs a 128×128 matrix multiplication.
- Matrix C is written back to memory.

Memory Map (DRAM)

Matrix A Start Address	:	BF000000
Matrix B Start Address	:	BF100000
Matrix C Start Address	:	BF200000

DRAM Address Range
80000000 – BFFFFFFF
(1 GB DRAM)

Send Linux Binary File to FPGA Board

After the bootloader appears in the Minicom terminal, transfer the Linux binary to the FPGA board using the provided send.sh script.

1 Execute the File Transfer Script

- Open a new terminal.
- Navigate to the file transfer script directory:
  - DVCon_2026/
    - FILE_TRANSFER/
      - send.sh
      - bbl.bin
      - bbl.dts
      - bbl.dtb
- Execute the send script:

```
./send.sh
```

user@linux: ~/DVCon_2026/FILE_TRANSFER

```
user@linux:~/DVCon_2026/FILE_TRANSFER$ ./send.sh
No of files: 2
address 1 : 0x80000000 [ ./bbl.bin ]
address 2 : 0x20000  [ ./bbl.dts ]
entry  : 0x80000000
entry  : 0x20000
Mac  : a022010180
Total files :2
File 1  : ./bbl.bin
File 2  : ./bbl.dts
Read count 111044696
------------------------------------------------------------------
--------------                          File checksum: db83d65f
File  : ./bbl.bin
Read count 111044696
File checksum: db83d65f
File 2  : ./bbl.dts
Read count 1460
File checksum: eea3

real    0m32.625s
user    0m0.611s
sys     0m0.288s
```

File transfer completed successfully.

2 Monitor Both Terminals

send.sh Terminal

```
No of files: 2
address 1 : 0x80000000 [ ./bbl.bin ]
address 2 : 0x20000  [ ./bbl.dts ]
entry : 0x80000000
entry : 0x20000
Mac  : a022010180
Total files: 2
File 1 : ./bbl.bin
File 2 : ./bbl.dts
Read count 111044696
------------------------------------------------------------------
--------------------          File checksum: db83d65f
File  : ./bbl.bin
Read count 111044696
File checksum: db83d65f
File 2 : ./bbl.dts
Read count 1460
File checksum: eea3

real    0m32.625s
user    0m0.611s
sys     0m0.288s
```

- Monitor transfer progress.
- Confirm successful completion.
- Check for transfer errors.

Minicom Terminal (Bootloader)

```
+-----------------------------------------------------------------+
|   VEGA Series of Microprocessors Developed By C-DAC, India      |
| Digital India RISC-V (DIR-V) Program, MeitY, Govt. of India     |
+-----------------+-----------+-----------------------------------+
|                 | -----     DVCON 2026      |                   |
+-----------------+-----------+-----------------------------------+
|     www.vegaprocessors.in       |          vega@cdac.in         |
+---------------------------------+-------------------------------+
MAC address     : 00:A0:22:01:01:80
Initializing ethernet...
Waiting for data...
     Download settings received

      Total files       : 2
      File 1 address    : 0x80000000
      File 2 address    : 0x20000
      Entry point       : 0x80000000

Waiting for file 1
```

- Verify the board receives and processes the binary.
- Watch for loading or boot errors.

3 Verify Successful Boot

/dev/ttyUSB0 - 115200 8N1 - Minicom

```
[    0.220053] rcu: Hierarchical SRCU implementation.
[    0.241342] smp: Bringing up secondary CPUS ...
[    0.247251] smp: Brought up 1 node, 1 CPU
[    0.267243] devtmpfs: initialized
[    0.287300] clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 7645041785100000 ns
[    0.290793] futex hash table entries: 16 (order: 2, 16384 bytes, linear)
[    0.809988] clocksource: timestamp_ns clocksource riscv_clocksource
[    0.806706] workingset: timestamp_bits=30 max_order=20 bucket_order=0 node
[    0.807700] serial: RISC-V SBI DBCN driver enabled
[    9.887990] Freeing unused kernel image (initmem) (size: 99 pages)
[   25.670006] Kernel memory protection not selected by kernel config.
[   25.680906] Run /init as init process
Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK

COAC Linux
cdac login:
```

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.8 | VT102 | Offline

- Linux binary loaded successfully.
- FPGA board is ready for use.
- Log in and continue development.

Login : root
Password : root

Linux successfully transferred and booted on the FPGA board.

4 Default Example Accelerator Output

```
Running sysctl: OK
Saving random seed: [   70.348000] random: dd: uninitialized urandom read (512 bytes read)
OK

C-DAC Linux
cdac login: root
Password:
login[107]: root login on 'console'
# ls
app_128
# ./app_128
DVCon Accelerator App
Data: 1
Result Generated
Result Matrix
128.000000
128.000000
128.000000
128.000000
128.000000
128.000000
128.000000
128.000000
128.000000
Data: 2
Result Generated
Result Matrix
512.000000
512.000000
```

- The default hardware accelerator performs 128×128 matrix multiplication.

Run the example application:

1. List the files in the current directory:

```
$ ls
```

Verify that the `app_128` executable is present.

2. Execute the application:

```
$ ./app_128
```

Your application is named **app_128**.

Application executed successfully.